

CS 6501 — Week 2 Notes: Safety and Control Barrier Functions

Big Picture

Week 2 introduces **Control Barrier Functions (CBFs)**, a formal framework for guaranteeing safety in robotic systems. The key idea is to enforce safety as a *constraint on control inputs*, rather than as a handcrafted avoidance behavior. This allows robots to pursue arbitrary objectives while remaining provably safe.

System Model

Robotic systems are modeled as continuous-time, nonlinear systems that are affine in control:

$$\dot{x} = f(x) + g(x)u$$

where:

- x is the system state (e.g., position, velocity),
- u is the control input,
- $f(x)$ captures natural system dynamics,
- $g(x)$ captures how controls influence motion.

The functions f and g are assumed to be Lipschitz continuous, ensuring that system trajectories exist and are unique. This is a technical assumption used to support theoretical guarantees.

Lyapunov Functions (Background)

Lyapunov functions provide a method to guarantee **stability**. Intuitively, a Lyapunov function behaves like an energy landscape:

- The function is minimized at a stable equilibrium.
- System dynamics drive the state downhill toward this minimum.

While Lyapunov methods ensure convergence or stability, they do not directly encode safety constraints.

Controllers via Optimization

Rather than prescribing a fixed control law, modern safety frameworks compute controls by solving an optimization problem at each time step. Typically, this takes the form of a **Quadratic Program (QP)**:

$$u^* = \arg \min_u \|u - u_{\text{ref}}\| \quad \text{s.t. constraints}$$

Here:

- u_{ref} is a nominal controller (e.g., go-to-goal),
- constraints enforce safety or performance requirements,
- the solution modifies the nominal control as little as possible.

This philosophy underlies both Control Lyapunov Functions and Control Barrier Functions.

Control Lyapunov Functions (CLFs)

Control Lyapunov Functions (CLFs) extend Lyapunov ideas by explicitly incorporating control inputs. A CLF ensures that there exists a control input that causes the Lyapunov function to decrease.

CLFs are primarily used to encode:

- goal-reaching,
- convergence,
- liveness or task completion.

Control Barrier Functions (CBFs)

Control Barrier Functions encode **safety**. A barrier function $h(x)$ defines a safe set:

$$\mathcal{C} = \{x \mid h(x) \geq 0\}$$

Interpretation:

- $h(x) > 0$: system is safely inside the allowable region,
- $h(x) = 0$: system lies on the safety boundary,
- $h(x) < 0$: unsafe state.

CBF Constraint

Safety is enforced by requiring the time derivative of the barrier function to satisfy:

$$\dot{h}(x, u) \geq -\alpha(h(x))$$

where $\alpha(\cdot)$ is a class- $\mathcal{K}$ function.

Intuitively:

- the system may approach unsafe regions,
- but not so quickly that it crosses the boundary,
- control actions are modified to prevent violation.

Safety Guarantee

A key theoretical result is:

If the system starts in the safe set $\mathcal{C}$ and the CBF constraint is satisfied at all times, then the system remains in $\mathcal{C}$ for all future time.

This property is known as **forward invariance** and represents a formal safety guarantee.

Geometric Intuition

Barrier functions can be visualized as geometric surfaces over the state space:

- safe regions correspond to higher values of $h(x)$,
- the boundary corresponds to $h(x) = 0$,
- approaching unsafe regions causes control to slow or reverse motion.

Rather than abruptly stopping the robot, CBFs smoothly modify behavior near boundaries, often resulting in oscillatory but safe motion near constraints.

Designing Barrier Functions

A major practical challenge is defining an appropriate barrier function. The choice of $h(x)$ is system-dependent and reflects what the designer considers unsafe, such as:

- distance to obstacles,
- inter-robot separation,
- altitude or workspace limits.

There is no universal barrier function for all robots or tasks.

CBFs in Multi-Robot Systems

In multi-robot settings, barrier functions are often defined pairwise between robots to enforce minimum separation. These ideas form the foundation for decentralized multi-robot safety methods studied later in the course.

Key Takeaways

- Safety is enforced as a constraint, not a behavior.
- Control inputs are minimally modified via optimization.
- CBFs provide formal, provable safety guarantees.
- Designing barrier functions is a core robotics challenge.